

AI-Based Assistive System for Correcting Dysgraphia and Dyscalculia in Children

Chekka Leela Prasanthi

Electronics and Communication

NIT Warangal, India

cl21ecb0a11@student.nitw.ac.in

Gandham Sushma

Electronics and Communication

NIT Warangal, India

gs21ecb0a15@student.nitw.ac.in

Prof. T. Kishore Kumar

Electronics and Communication

NIT Warangal, India

kishoret@nitw.ac.in

Abstract—Dysgraphia and dyscalculia make it to the list of learning disabilities that are most frequently noticed in students as a significant hindrance towards their academic performance. The aim of this research is to formulate strategies for the specific difficulties faced by students suffering from dysgraphia and dyscalculia. Dysgraphia students' written sentences which contain grammatical and spelling errors were reconstructed using the T5 transformer model. Similarly, the students' works were analyzed through the use of BERT model which focused on mathematical reversals, operator, and other errors made by the dyscalculic students. In a similar way a program is developed in python to assess and predict the types of errors based on the values obtained from the BERT model. Evidence from this research demonstrates how everything is about transformer models these days, as they can actually allow you to create practical assets and adopt learning styles suitable for students with learning disabilities.

Keywords— *Dysgraphia; Dyscalculia; T5 Transformer model; BERT model; SLD*

I. INTRODUCTION

Dysgraphia and dyscalculia are common Specific Learning Disabilities (SLDs), occurring in 5-7% of school-aged children and having a critical on academic performance and emotional well-being. Dysgraphia is a neurological condition affecting writing, spelling, and thought on paper. It can be further classified into motor dysgraphia, where students struggle with hand control needed for writing (leading to messy, hard-to-read handwriting), linguistic dysgraphia (affecting spelling and thought organization), and spatial dysgraphia (resulting in uneven spacing or misaligned text, though spelling and language skills remain intact).

Dyscalculia, on the other hand, impairs a child's ability to understand numbers and perform calculations. It includes quantitative dyscalculia, where students struggle with number understanding and relationships, and lexical (verbal) dyscalculia, where difficulties arise with mathematical language and word problems due to language processing issues.

Existing methods for addressing these disabilities are time-consuming and resource-heavy, delaying critical support and negatively impacting academic progress. Recent advances in artificial intelligence (AI), particularly in natural language processing (NLP), offer opportunities to improve these methods. This research aims to develop an AI-powered assistive system that helps students with SLDs by identifying and correcting their errors.

The system utilizes the T5 transformer model to resolve spelling and grammatical errors in dysgraphic students'

writing, and the BERT model to identify and classify mathematical errors in dyscalculic students' work. However, accurately identifying specific types of errors in both writing and mathematical reasoning remains a challenge. This research aims to improve students' abilities by pointing out spelling mistakes, suggesting corrections, and improving mathematical understanding.

A. Related Work

Recent developments in NLP have transformed rule-based arithmetic problem-solving to more flexible data-driven approaches. Neural networks help identify cognitive patterns and map variables dynamically to form equations. Whereas BERT and other similar transformer models enable a deeper understanding of word problems due the attention heads and positional encoding, while this combination has improved accuracy in solving arithmetic word problems, as shown by models achieving 91.1 percent accuracy on addition-subtraction tasks and 75.5 percent on mixed operations [1].

As BERT's general purpose pre-training has the potential to lead to mismatched word vectors for math problems, new strategies like task-specific fine-tuning have been established to align word vectors more accurately with the problem needs. Loss-priority self-supervised pre-training is another technique that facilitates faster model convergence by giving preference to data of higher losses. On the other hand, models that used analogy-based training have shown great performance on standard math problem datasets compared to other state-of-the-art approaches [2].

NLP models, especially the T5 architecture, have demonstrated state-of-the-art performance across various functions by employing a two-step process: self-supervised pre-training on large, general datasets and fine-tuning on smaller, task-specific datasets. Researchers have begun applying T5 to NLP tasks by pre-training it on combined natural language and source code data, then fine-tuning it for specific objectives like bug fixing and assert generation. Compared to conventional deep learning models T5 has shown better performance, indicating its strong potential for diverse NLP based tasks[3].

Literature in this field increasingly explores digital diagnostic tools and therapeutic activities, as informatics-based solutions offer enhanced accuracy, efficiency, and accessibility in identifying and supporting individuals with dyscalculia, making early intervention and tailored therapy more effective[4].

Current studies of developmental dysgraphia emphasize the application of digital devices, e.g., digitizing tablets, to record handwriting kinematics with measures of pen pressure, tilt, and stroke patterns for writing disability diagnosis in children. Machine learning algorithms such as random forests have proven to be highly accurate in dysgraphia detection, with studies attaining sensitivity and specificity levels of up to 96 percent. Further, automated tests using the handwriting proficiency screening questionnaire (HPSQ) provide robust severity estimation, enabling early and scalable intervention programs [5].

B. Existing Work

Previous research focused on a wide range of approaches for grammar and spelling correction. For example, the first approach utilizes multi-feature fusion with transformer models like ALBERT and mBERT, which contain part-of-speech tagging and tonal information to enhance syllable representation, even though this technique can be computationally extensive [6]. An NLP-based system is developed for Arabic grammar correction which generates syntactically correct sentences by analyzing and reconstructing occurrences [7], while another research is based on this which uses unsupervised learning and increases training data to improve accuracy in Arabic script correction [8]. Many Deep learning models are explored in machine translation to increase the error correction continuously, and this research achieved high accuracy in identifying and correcting mistakes [9]. Another research used LSTM models for Chinese pinyin spelling correction, by using a combination of character, text, and pinyin features which is designed to handle various spelling variations in Chinese text. Other research includes analyzing language data to identify common vowel and consonant errors made by non-native speakers of English and Spanish [6].

C. Proposed Work

In this research, we fine-tuned the T5 model using a relatively small, synthetic dataset to create a straightforward and effective method for fixing spelling and grammar errors, which are common challenges in dysgraphia. The existing works used neural architectures and included multi-feature extraction to achieve high accuracy, but they require a lot of computational power and large training datasets. Hence, we aimed to provide a solution that can balance easy implementation and acceptable accuracy, making it particularly useful in scenarios where rapid deployment or resource constraints are major issues. To enhance the error detection and correction capabilities, we developed a hybrid system using BERT for classification and SymPy for symbolic validation of multi-step mathematical expressions, which is beneficial for addressing dyscalculia. This system detects dyscalculic errors such as number reversal and

operator misinterpretation by evaluating each step of the expression and identifying discrepancies.

For dysgraphia, we created a prototype app that uses the T5 model in the backend to correct spelling and grammar mistakes. This app serves as a practical demonstration of how T5 can assist individuals with dysgraphia by providing real-time spelling and grammar corrections, making it easier for users to produce error-free written text.

While the BERT + SymPy pipeline successfully detects and validates dyscalculic errors, natural language explanations of those errors were not fully realized within the symbolic system. To address this, we prototyped the use of large language models (LLMs) like Bard to generate human-readable explanations. For example, the system can convert symbolic error patterns into sentences like "12 -> 21 due to reversal" to explain where the user made an error. To demonstrate this, we created a prototype app that showcases how the BERT + SymPy system detects errors and how Bard generates corresponding natural language explanations. This app provides a tangible preview of how users would receive real-time feedback, combining error detection with easy-to-understand explanations.

However, this explanation generation component is currently separate and serves as a proof-of-concept. The integration of Bard or similar LLMs into the core BERT + SymPy pipeline is a key future direction for enhancing interpretability and providing personalized feedback, which would make the system more user-friendly and impactful in assistive educational contexts.

II. METHODOLOGY

This research presents a hybrid system combining T5, BERT, SymPy, and Bard to address dysgraphia and dyscalculia by providing real-time error detection, correction, and feedback with a balance of computational efficiency and accuracy. For dysgraphia, the T5 model is fine-tuned on a small, synthetic dataset to correct spelling and grammar errors. The T5 model architecture is illustrated in Figure 1, using its encoder-decoder architecture, processes input sequences with errors and generates corrected output. A prototype app is developed to offer real-time feedback, assisting individuals with dysgraphia in producing error-free text. For dyscalculia, BERT model is combined with SymPy to detect errors in mathematical expressions and the architecture is shown in Figure 2. BERT's bidirectional attention and positional encoding help interpret expressions, while SymPy performs symbolic validation of multi-step calculations. This hybrid system detects issues like number reversal and operator misinterpretation, providing accurate feedback for dyscalculic errors.

To improve user understanding, Bard, an LLM, is integrated to generate natural language explanations for errors detected by BERT and SymPy. For example, Bard explains a

reversal error as "Digits were reversed, 12 became 21," enhancing the system's interpretability. An app is prototyped integrating BERT + SymPy for error detection and Bard for generating explanations. While this explanation generation is separate, future work will focus on fully integrating Bard into the core pipeline for real-time personalized feedback.

User feedback from individuals with dysgraphia and dyscalculia is collected to assess accuracy, response time, and user satisfaction, ensuring that the system is both effective and easy to use. In conclusion, the hybrid system combining T5, BERT, SymPy, and Bard offers an efficient solution for real-time error correction and feedback in educational contexts, especially where resource constraints or the need for rapid deployment exist.

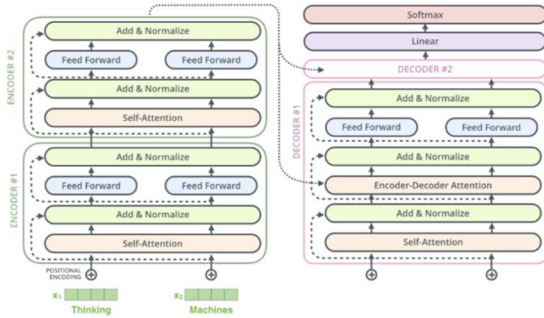


Figure 1. Figure showing the detailed architecture of T5 transformer

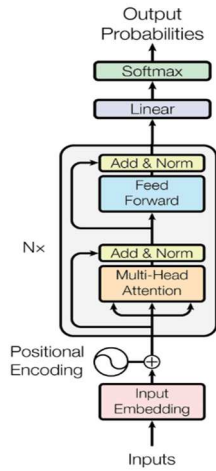


Figure 2. Figure showing the detailed architecture of BERT transformer

A. Dataset

To analyze the problem-solving behavior of students with dyscalculia, we developed a synthetic dataset that simulates common mathematical errors made by children aged 7–9. Each student in the dataset is assigned a unique Student ID and attempts ten basic arithmetic questions, resulting in a total of 10,000 samples. The dataset is designed to reflect a wide range of error types to facilitate effective

training and evaluation of machine learning models for error pattern recognition. The dataset includes the following key features: Student_ID (unique identifier), Incorrect_Result (student's response), Correct_Result (accurate solution), and Error_Types (categorized error type). The error types include Number Reversal (e.g., 23 instead of 32), Operator Misinterpretation (e.g., treating ' $\times$ ' as '+'), BODMAS Misapplication (incorrect order of operations), and None (correct response). The mathematical expressions are simple (e.g., " $25 + 27$ ", " $27 / 3 + 4$ ") and tailored to the skill level of elementary students. The dataset ensures coverage of all error type combinations, making it a robust foundation for training models that can detect and categorize dyscalculic error patterns.

B. Implementation Flow

The implementation starts with data collection, where the dataset is organized into a CSV format containing two columns: one for Input, consisting of text with spelling and grammatical errors, and the other for the Target, containing the corrected version of the input text. A total of 10,000 samples are used, covering a wide range of examples for training. OCR processing follows, where handwritten text is converted into digital text using pre-trained models like Tesseract. This step reduces noise and enhances character recognition accuracy, ensuring clean digital text for further analysis. The input formatting for the T5 model involves preprocessing the text by removing special characters, standardizing the case, and structuring the text to clearly specify the correction task (e.g., "correct: The cta is on th mat." to "The cat is on the mat"). This is essential for guiding the model's learning process.

Next, fine-tuning the T5 model is performed to correct spelling and grammar using the dysgraphic dataset and the pseudocode is shown in Figure 3, flowchart is shown in Figure 6. Using the Hugging Face Transformers library, the model is trained with a batch size of 8, a learning rate of $5e-5$, the AdamW optimizer with a weight decay of 0.01, cross-entropy loss, logging every 10 steps, and gradient clipping. After training, the T5-small model shows promising results, with a training loss of 0.0001 and a validation loss of 0.00018, indicating effective learning and correction accuracy. For dyscalculia, the flow is adapted to detect and classify errors in mathematical expressions. The dataset for training consists of mathematical expressions, student responses, correct answers, and error annotations like number reversal and operator misinterpretation. The input is tokenized, and the BERT model is fine-tuned on this labeled dataset. Training parameters include 4 epochs, a batch size of 16 for training, and 64 for evaluation, with the AdamW optimizer and binary cross-entropy loss for multi-label classification. The model is evaluated using accuracy, precision, recall, F1-score, and confusion matrix analysis. The model predicts error types, such as number reversal, operator misinterpretation, and BODMAS misapplication, based on tokenized expressions.

The error detection in mathematical expressions involves processing the tokenized expressions using the fine-tuned BERT model, which outputs probabilities for each error type. The predictions are compared with predefined thresholds to identify specific errors. The results for each error type—

number reversal, operator misinterpretation, and BODMAS misapplication—are printed based on the model's output, guiding the process of error identification and classification. Additionally, the system provides step-by-step evaluations by parsing mathematical expressions into an expression tree, where operators form internal nodes and numbers are the leaves. As the tree is constructed, subexpressions are recursively evaluated, recording each intermediate result. This method enables students to understand where they made mistakes and grasp the correct order of operations, fostering better comprehension of the problem-solving process. The pseudocode of this process is described in Figure 4, 5 and the flowchart is shown in Figure 7.

To integrate these technologies into a practical solution, we developed a prototype app for both dysgraphia and dyscalculia. For dysgraphia, the app uses the T5 model for real-time spelling and grammar correction. For dyscalculia, the app incorporates the BERT + SymPy system for error detection and feedback, along with a feature to generate natural language explanations using Bard, an LLM, to explain identified errors in a human-readable format. This app serves as a demonstration of how the system can assist individuals with dyslexia and dyscalculia by providing real-time feedback and clear explanations.

The overall system integrates error detection with educational support, offering students a structured breakdown of their mistakes in mathematical expressions and spelling/grammar errors. The approach aims to enhance their problem-solving skills, confidence, and understanding, making the system a valuable tool for educational support in real-world applications.

Algorithm: Dysgraphia Text Corrector

1. Start

2. Dataset Generation:
 - Create synthetic dataset with sentences using predefined templates and vocabulary.
 - Introduce random errors (phonetic mistakes, spacing issues, letter reversal, etc.) to simulate dysgraphia
 - Label dataset with severity and error types (phonetic, letter reversal, etc.).

3. Data Preprocessing:
 - Convert the dataset into a Hugging Face Dataset format.
 - Tokenize the input_text and corrected_text using the T5 tokenizer.
 - Ensure tokenized input has padding and truncation with a max length of 512 tokens.

4. Model Setup:
 - Load a pre-trained t5-small model and its tokenizer.
 - Prepare the dataset by splitting it into training (75%) and validation (25%) sets.

5. Training Setup:
 - Set up training arguments (batch size, number of epochs, evaluation strategy).
 - Define a Trainer using the model, training arguments, and dataset.

6. Training:
 - Train the model over multiple epochs (6 in this case).
 - Track and log training and validation losses.

7. Evaluation:
 - Use the trained model to evaluate performance on the validation dataset.
 - Optionally, compute evaluation metrics like BLEU score to assess correction quality.

7. End

Figure 3. Pseudo code of Dysgraphia Text Correction

Algorithm: Identify and Classify Errors in Mathematical Expressions for Dyscalculia

1. Start

2. Preprocess Input Data:
 - For each student response tokenize the mathematical expression and labels

3. Fine-tune BERT Model:
 - Load pre-trained BERT model and tokenizer.
 - Fine-tune BERT on the dataset containing:
 - Mathematical expressions, Annotated error types
 - Set training parameters:
 - Learning rate = 1e-4, Batch size = 8, Epochs = 3
 - Evaluate model using accuracy, confusion matrix.

4. Error Classification and Detection:
 - For each student expression:
 - Pass tokenized expression through the fine-tuned BERT model.
 - Retrieve output probabilities for each error type

5. Error Thresholding:
 - Define F1-score threshold (e.g., 0.5) for error classification.
 - If (Error Probability > Threshold):
 - Identify potential error: - Check and log error based on the model's prediction:
 - If the error corresponds to a number reversal, log as "Potential error: Number Reversal"
 - If the error corresponds to operator misinterpretation, log as "Potential error: Operator Misinterpretation"
 - If the error corresponds to BODMAS misapplication, log as "Potential error: BODMAS Misapplication"

6. No Error Detected:
 - If no error probabilities exceed threshold, print "No errors detected."

7. Print Results

8. End

Figure 4. Pseudo code of Identify and classify errors in mathematical expression for Dyscalculia

Algorithm: Step by Step Evaluation Process of an Expression

1. Start

2. Input Preprocessing:
 - Process expression by removing spaces and recognizing numbers and operators.

3. Construct Expression Tree:
 - Traverse the expression to build a tree using stacks for values and operators.
 - For each operator or operand, build subtrees and update the stack.

4. Evaluate Tree:
 - Recursively evaluate the tree:
 - If leaf node, return its value.
 - Otherwise, evaluate left and right subtrees and apply the operator.

5. Step-by-Step Evaluation:
 - For each evaluation, store and print Step x: expression = result.

6. End

Figure 5. Pseudo code of Step-by-Step Evaluation Process of an expression

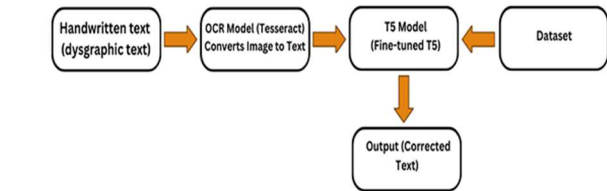


Figure 6. Flowchart of the Solution for Correcting Mistakes in Dysgraphia Text

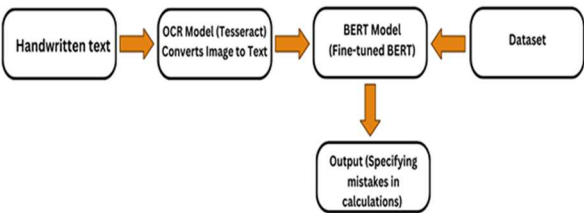


Figure 7. Flowchart of the Solution for Correcting Mistakes in Dyscalculia Patients

III. RESULTS

In this study, we developed an app prototype to assist students with dyscalculia and dysgraphia by providing real-time error detection and correction. For dysgraphia, the app uses a T5 model to process student-written text and identify and correct spelling and grammatical errors. For example, given the input "this sentnce has a couple with errors," the app first corrects the spelling to "this sentence has a couple with errors," followed by a grammatical correction to "This sentence has a couple of errors." Similarly, for the input "I have an speling error in this sentence," the model initially corrects the spelling to "I have a spelling error in this sentence," then applies a grammatical correction to "I have a spelling error in this sentence." These results demonstrate the model's effective ability to refine both spelling and grammar errors and shown in the Figure 9.

For the dyscalculia model, we used a combination of BERT and SymPy to provide feedback by addressing specific mathematical miscalculations. For instance, when the input is "39 + 12 / 3" and the student's answer is "75," the model identifies potential number reversals (e.g., interpreting 39 as 93 and 12 as 21) and operator misinterpretations (e.g., mistaking "+" for "-" and "/" for "+"). This process led to the incorrect evaluation of "93 - 21 + 3" in Figure 12. The model's accuracy is high, with precision rates of 0.97 and 0.98. However, its performance is lower when no errors were present, with a recall of 0.24, suggesting that there is room for improvement in detecting error-free expressions. Training results for both models indicated effective learning. The T5 dysgraphia model showed a steady reduction in both training and validation losses over three epochs, reaching a final training loss of 0.2551 and a validation loss of 0.0410 is shown in Figure 8. The model achieved a training speed of 10.25 samples/sec with a computational cost of 3.04×10^{15} FLOPs. The BERT+SymPy dyscalculia model, while effective in recognizing errors, demonstrated room for improvement in handling error-free expressions, as shown in the confusion matrix in the Figure 11. Initially the evaluation loss increases with epochs which represents the training phase as the parameters are not optimized which leads to the overfitting in Figure 10. As the epochs further increase, the model adjusts its parameters to minimize the evaluation loss. Figure 13, shows the step-by-step process of evaluating the given expression. Figure 11's confusion matrix further illustrates the model's classification accuracy across error categories, with an evaluation loss of 0.1938, underscoring its reliability in analyzing expressions and recognizing dyscalculic patterns, enabling targeted educational support. The prototype app home page is shown in Figure 14, and is able to correctly identify and address errors, offering targeted educational support. For example, it corrected a final answer of 361 to the correct value of 90.25, and the process of uploading the input is shown in Figure 18, and the result is shown Figure 19, showcasing its ability to provide real-time, accurate feedback for students with dyscalculia. Similarly, for dysgraphia, the app successfully transformed the student's written text into grammatically correct and properly spelled sentences, demonstrating its effectiveness in assisting students with writing challenges and the process of uploading input and getting output is shown in Figure 15, 16 and 17.



Figure 8. Graph showing the Training and Validation loss for each epoch of T5 model for dysgraphia

Spell-corrected text: this sentence has a couple with errors
Grammar-corrected text: This sentence has a couple of errors.

Figure 9. Output of T5 Model: Corrected Spelling and Grammar for Dysgraphic Text

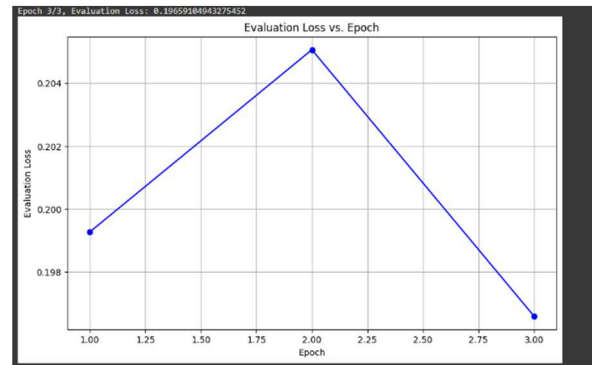


Figure 10. Graph showing variation in evaluation loss with each epoch of BERT model

	precision	recall	f1-score
BODMAS Misapplication	0.76	0.88	0.81
None	0.93	0.21	0.35
Number Reversal	0.94	0.87	0.90
Operator Misinterpretation	0.96	0.88	0.92

Figure 11. Confusion Matrix of BERT model

Expression: 39 + 12 / 3
Student's Result: 75
Errors Detected:
Assumed Reversal: 39 -> 93; 12 -> 21 and Operator Misinterpretation: + -> -; / -> + resulted in '93 - 21 + 3'

Figure 12. Output of the BERT and SymPy Model

```

expression = "(3 + 5) * (2 + 3) - (4 / 2) + (7 + 6 * 2) / (5 + 3)"
Evaluation Steps:
Step 1: (3 + 5) = 8
Step 2: (2 + 3) = 5
Step 3: ((3 + 5) * (2 + 3)) = 40
Step 4: (4 / 2) = 2.0
Step 5: (((3 + 5) * (2 + 3)) - (4 / 2)) = 38.0
Step 6: (6 * 2) = 12
Step 7: (7 + (6 * 2)) = 19
Step 8: (5 + 3) = 8
Step 9: (((7 + (6 * 2)) / (5 + 3)) - 2.375)
Step 10: (((((3 + 5) * (2 + 3)) - (4 / 2)) + ((7 + (6 * 2)) / (5 + 3)))) = 40.375

```

Figure 13: Step-by-step evaluation of given input expression

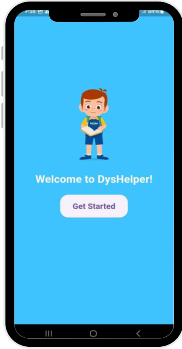


Fig 14: Home Page

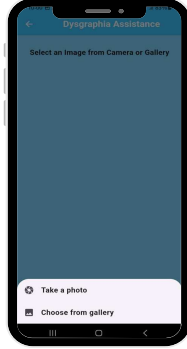


Fig 15: Dysgraphia uploading image

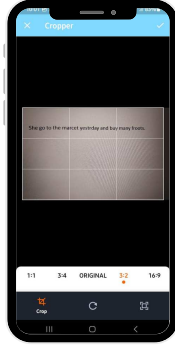


Fig 16: Image Cropping



Figure 17: Dysgraphia output

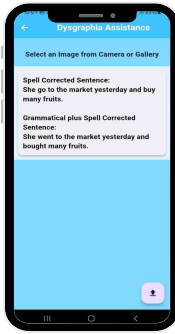


Figure 18: Dyscalculia Input uploading

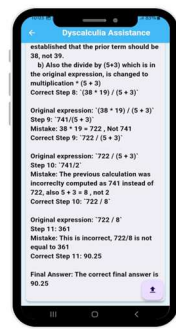
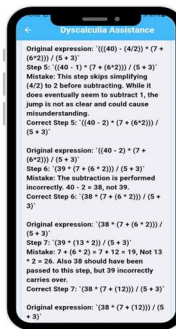
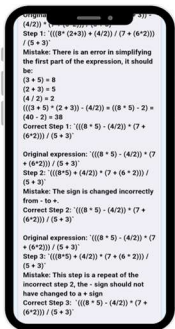


Figure 19: Dyscalculia output containing errors and correct procedure of evaluating each step

IV. CONCLUSION AND FUTURE WORK

Future Future work will focus on expanding the dataset with diverse, real-world samples to improve the generalization of models tailored for students with dysgraphia and dyscalculia. For dysgraphia, the T5-based app can be further refined with real-time deployment and continuous learning from user interactions to enhance spelling and grammar correction.

For dyscalculia, the BERT + SymPy core pipeline—used for symbolic error classification and mathematical expression validation—can be strengthened by incorporating more complex arithmetic patterns and edge-case errors. Integration of large language models like Bard into this pipeline is a key direction, aiming to translate symbolic errors into natural language explanations and make the feedback more understandable and personalized. Additionally, incorporating adaptive learning mechanisms and supporting multimodal inputs such as audio and handwriting will further improve accessibility. These advancements will help create a comprehensive assistive tool capable of delivering real-time, intelligent feedback in educational settings.

VI. REFERENCES

- [1] R. Peng, X. Lyu and X. Yu, "Arithmetic Problem Solver Based on BERT Model and Mathematical Cognitive Pattern," 2021 IEEE International Conference on Engineering, Technology & Education (TALE), Wuhan, Hubei Province, China, 2021, pp. 834-837, doi: 10.1109/TALE52509.2021.9678839.
- [2] Y. Jia, P. Wang, Z. Zhang, C. Cheng, Z. Li and X. Yu, "A BERT-Based Pre-Training Model for Solving Math Application Problems," 2022 International Conference on Intelligent Education and Intelligent Research (IEIR), Wuhan, China, 2022, pp. 71-77, doi: 10.1109/IEIR56323.2022.10050073.
- [3] A. Mastropaolo et al., "researching the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks," 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), Madrid, ES, 2021, pp. 336-347, doi: 10.1109/ICSE43902.2021.00041.M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.
- [4] F. Ferraz and J. Neves, "A brief look into dyscalculia and supportive tools," 2015 E-Health and Bioengineering Conference (EHB), Iasi, Romania, 2015, pp. 1-4, doi: 10.1109/EHB.2015.7391584.
- [5] J. Mekyska, M. Faundez-Zanuy, Z. Mzourek, Z. Galaz, Z. Smekal and S. Rosenblum, "Identification and Rating of Developmental Dysgraphia by Handwriting Analysis," in IEEE Transactions on Human Machine Systems, vol. 47, no. 2, pp. 235-248, April 2017, doi: 10.1109/THMS.2016.2586605.
- [6] Moukrim C, Abderrahim T, Tarik A. An innovative approach to auto correcting grammatical errors in Arabic texts[J]. Journal of King Saud University-Computer and Information Sciences, 2021, 33(4): 476-488.
- [7] Solyman A, Zhenyu W, Qian T.Synthetic data with neural machine translation for automatic correction in arabic grammar[J]. Egyptian Informatics Journal, 2021, 22(3): 303-315.
- [8] Wang H, Wang B, Duan J.Chinese spelling error detection using a fusion lattice LSTM[J]. Transactions on Asian and Low-Resource Language Information Processing, 2021, 20(2): 1-11.
- [9] Lindner A L, Wijekumar K, Joshi R M. English spelling performance in writing samples among Spanish-speaking ELLs[J]. Journal of Learning Disabilities, 2022, 55(2): 114-122.